



SMART CONTRACT SECURITY ASSESSMENT

PREPARED FOR

GOLDBACKBOND CONTRACTS



hello@interfi.network



interfinetwork



www.interfi.network

FOR GOLDBACKBOND

Date of Report	January 05, 2026
Scope	Token - GOLDBACKBONDBase Mint- USDGBMinting Stake - CertificateStaking Vault - GoldBonusVault Admin - Guardian
Platform	Base Sepolia Testnet
Website	
Language	Solidity
Methodology	Automated Review, Unit Tests, Manual Review
Auditor	InterFi

- Disclaimer: Smart contracts deployed on blockchains are inherently exposed to potential exploits, vulnerabilities, and security risks. Blockchain and cryptographic technologies are emerging and carry ongoing uncertainties. Please review the full audit report for detailed insights into risk severity, vulnerabilities, and audit scope limitations.
- Centralization Warning: Centralized privileges—regardless of intent or access control—introduce elevated risks to contract security and user trust.
- KYC Advisory: The project lacks verified third-party KYC of its owners, team, or deployers. Without independent KYC, transparency and accountability are reduced, increasing the risk of fraud or rug pulls.
- Verification: Verify this report: <https://www.github.com/interfinetwork>

TABLE OF CONTENTS

FOR GOLDBACKBOND	1
TABLE OF CONTENTS	3
1. SUMMARY	4
1.1 Summary of Findings	4
1.2 Resolution Status	4
1.3 System Overview	5
1.4 Files in Scope	5
1.5 Out-of-Scope Assumptions	6
2. METHODOLOGY	7
2.1 Audit Objectives	7
2.2 Methodologies	7
2.3 Risk Categorization	8
2.4 Resolution Status Definitions	9
3. FINDINGS	10
4. CENTRALIZATION	26
4.1 Privileged Functions	26
5. DISCLAIMER	35
5.1 Confidentiality	35
5.2 No Financial Advice	35
5.3 Technical Disclaimer	35
5.4 Timeliness & Accuracy	36
5.5 Third-Party Links	36
6. ABOUT	37
6.1 Connect with Us	37

1. SUMMARY

The audit resulted in the identification of issues across a range of severity levels, including logic flaws, access control oversights, and design inconsistencies. All high-impact findings were communicated to the development team with clear recommendations for remediation. Where applicable, the team has confirmed implementation of fixes or provided justifications for design choices.

1.1 Summary of Findings

Severity	Count
Critical	1
Major	6
Medium	6
Minor	2
Unknown	1
Centralization	6

1.2 Resolution Status

Status	Count
Fixed	0
Partially Fixed	0
Acknowledged	0
Pending	22

1.3 System Overview

This system is a decentralized protocol comprising a suite of smart contracts. These contracts collectively define the rules, permissions, and operational workflows for managing on-chain assets, executing user interactions, and enforcing protocol-level logic. Smart contracts in this context are self-executing code units that autonomously manage the state and behavior of digital assets based on predefined conditions. The protocol utilizes these contracts to enable key functionalities such as:

- Ownership and access control enforcement
- Permission and role-based actions
- Data storage and updates
- Event logging and auditability
- Batch processing and collection management

1.4 Files in Scope

InterFi was engaged by Goldbackbond to perform a security audit of the smart contracts. The audit scope was strictly limited to the files explicitly listed under the “Files in Scope” section. No other files or components were reviewed unless otherwise stated.

File	Path	Notes
Token	../GOLDBACKBONDBase	Custom Module
Mint	../USDGBMinting	Custom Module
Stake	../CertificateStaking	Custom Module
Vault	../GoldBonusVault	Custom Module
Admin	../Guardian	Custom Module

1.5 Out-of-Scope Assumptions

The following components and assumptions were explicitly excluded from this audit:

- Frontend or backend integration logic.
- Off-chain components, scripts, or oracles.
- External contracts or libraries unless explicitly stated.
- Compiler-level or EVM-specific behavior outside the contract's scope.
- Governance or tokenomics-related decisions not implemented in code.
- All third-party dependencies as discussed in findings.

2. METHODOLOGY

2.1 Audit Objectives

This audit aims to ensure that the smart contract system is predictable, and behaves as intended under normal conditions. Primary audit objectives are to:

- Identify potential vulnerabilities or logic errors in the implementation.
- Evaluate adherence to best practices in smart contract development.
- Assess the correctness of access controls and permission systems.
- Recommend remediations or enhancements for improved security and performance.

2.2 Methodologies

The audit follows a layered security approach using both automated tools and manual techniques. We review the contracts for functional correctness, exploitability, and adherence to smart contract best practices:

Type	Tools & Techniques
Manual Code Review	Line-by-line analysis to check logic, permissions, and edge cases
Automated Analysis	Tools like Slither, MythX, or custom linters to catch known patterns
Static Analysis	Identification of bugs without executing the code (compile-time checks)
Unit Test Inspection	Evaluation of existing test coverage, assumptions, and potential false positives/negatives (if applicable)
Architecture Review	Mapping of privileged roles, callable paths, and contract interdependencies (if applicable)

2.3 Risk Categorization

Each issue identified during the audit is assigned a severity level based on its potential impact, exploitability, and likelihood of real-world abuse. These categories help prioritize remediation efforts:

Risk Severity	Definition
Critical	Represents a severe vulnerability that may result in complete contract compromise, such as asset theft, permanent loss of functionality, or unrestricted access. These issues are often easily exploitable and require immediate resolution.
Major	Indicates significant risk that can affect core contract behavior, enable unauthorized operations, or create unintended financial exposure/loss. While not as urgent as critical risks, they should be remediated promptly.
Medium	These are moderate-level risks that may become exploitable under specific conditions. They often relate to logic errors, insufficient validation, or architectural oversights that could escalate over time.
Minor	Denotes issues that have low security impact but may degrade code quality, performance, or maintainability. These include inefficiencies, style violations, or redundant logic. Fixes are recommended for robustness.
Unknown	Risks where the severity cannot be confidently determined due to limited context, external dependencies, or ambiguous design intent. It is advised to treat these conservatively and address them proactively.
Centralization	Any function controlled by a single privileged role is treated as a critical risk, regardless of its purpose, due to the potential for misuse, override, or total asset control.

2.4 Resolution Status Definitions

All identified issues are also assigned a resolution status, indicating the current handling and response from the development team:

Status	Definition
 Fixed	The issue has been remediated and verified as resolved during the re-audit or final check.
 Partially Fixed	The issue has been partially mitigated, but remnants or related concerns may still exist. Further attention may be required.
 Acknowledged	The development team has accepted the finding but opted not to implement a fix.
 Pending	The issue remains unresolved at the time of publication. It poses a potential risk and should be addressed.

3. FINDINGS

01	LP withdrawals can be permanently blocked when reward pool is empty
Severity	Critical
Affected	LpRewardPool.withdraw(), LpRewardPool.stake(), LpRewardPool.claimReward(), LpRewardPool._claimReward()
Description	Both withdraw() and stake() call _claimReward() first. _claimReward() hard-reverts if <code>reward > GBB_TOKEN.balanceOf(address(this))</code> with "Reward pool empty". If the contract's GBB reward balance is insufficient (whether due to underfunding, migration, accidental transfer-out, or reward depletion), users cannot withdraw their LP tokens because the withdrawal path reverts before <code>LpAmount</code> is reduced and before LP is transferred back. This is a loss-of-access / fund-lock condition for user LP.
Recommendation	Decouple LP withdrawals from reward availability
Status	 Pending

O2	MAX_SUPPLY is enforced per-chain, not globally across OFT mesh
Severity	Major
Affected	GOLDBACKBONDBase._update() OFTCore._lzReceive() → OFT._credit() → ERC20._mint() → GOLDBACKBONDBase._update()
Description	MAX_SUPPLY check in _update() correctly caps the local chain's totalSupply() on mint operations, including LayerZero credit-mints (_credit() calls _mint()). However, for OFT meshes deployed on multiple chains, this cap is only enforced on each chain independently. If minting is enabled on more than one chain (or if a trusted remote can cause inbound minting), the aggregate supply across all chains can exceed MAX_SUPPLY even though each individual chain remains under the cap. This is a common OFT foot-gun: “global cap” cannot be enforced by checking totalSupply() on each chain.
Recommendation	If the intent is a global cap, restrict minting to a single “canonical mint chain” and disallow privileged minting on other chains, or enforce a global cap via a cross-chain accounting mechanism (canonical supply chain, verified supply messages, or an external cap oracle/registry). At minimum, explicitly document that MAX_SUPPLY is a per-chain cap unless additional controls exist off-chain or at the protocol level.
Status	Pending

03	Chainlink price feed values are used without safety checks Oracle safety assumptions are not enforced - potential flash loan risk
Severity	Major
Affected	GoldBonusVault.constructor, GoldBonusVault.getLatestGoldPrice(), GoldBonusVault.distributeBonus()
Description	latestRoundData() is consumed without validating: - price > 0 (an int256 is cast to uint256; a negative value becomes a huge uint256), - updatedAt recency (stale data), - answeredInRound >= roundId integrity, - baselineGoldPrice != 0 (division by zero in percentIncrease if baseline is 0). These omissions can cause: - a permanent DoS of distributeBonus() via division by zero, - unintended bonus distributions if the oracle returns anomalous values. - external manipulation to drain vault rewards. Since distributeBonus() transfers real tokens from the vault, oracle anomalies translate into direct financial exposure.
Recommendation	Add oracle validation: require price > 0, require updatedAt != 0 and block.timestamp - updatedAt <= maxStaleness, require answeredInRound >= roundId, and guard baseline initialization against 0.
Status	Pending

04 Role-admin hierarchy is bypassed via custom grant/revoke wrappers	
Severity	Medium
Affected	GOLDBACKBONDBase.grantMinterRole(), revokeMinterRole(), grantBurnerRole(), revokeBurnerRole()
Description	Contract introduces an ADMIN_ROLE and then uses onlyRole(ADMIN_ROLE) wrappers that call _grantRole() / _revokeRole() directly. This bypasses AccessControl’s built-in getRoleAdmin(role) authorization flow.
Comment	Either set MINTER_ROLE/BURNER_ROLE admin to ADMIN_ROLE using _setRoleAdmin() and use standard grantRole/revokeRole, or keep wrappers but explicitly document that ADMIN_ROLE is an out-of-band admin for these roles, and harden governance ADMIN_ROLE membership with multi-sig. (As discussed in Centralization-01)
Status	Pending

O5	USDGBMinting queue enables unbounded storage growth and operational DoS
Severity	Medium
Affected	USDGBMinting.mint(), USDGBMinting.complianceQueue, USDGBMinting.queueNonce
Description	Any address with MINTER_ROLE can enqueue an unbounded number of “yellow lane” transactions (<code>amount > softLimit && amount < hardLimit</code>). These entries are never deleted, only marked <code>processed = true</code> . Over time, this creates permanent state growth (higher gas costs for reads, higher operational burden for compliance review, and potential block gas / indexing issues for off-chain systems). While this is “by a privileged role,” it is still a practical foot-gun and a realistic failure mode in compromised-minter scenarios.
Comment	Implement bounded queues and/or deletion on completion (e.g., delete storage after approve/reject). Consider rate limits per minter, maximum pending items, or off-chain queueing with on-chain execution proofs.
Status	Pending

06	GoldBonusVault distribution loop is unbounded and can become non-executable
Severity	Medium
Affected	GoldBonusVault.distributeBonus()
Description	distributeBonus() iterates over users[] with no upper bound. If the array is large, the function will exceed the block gas limit and become uncallable, preventing bonus distribution. Because the “batching” strategy is only documented in comments, the contract itself does not enforce safe batch sizing or provide pagination helpers.
Comment	Enforce a maximum users.length per call and/or implement a paginated distribution mechanism using stored cursor state.
Status	Pending

07	GoldBonusVault lacks reentrancy protection around token transfers USDGBMinting lacks reentrancy protection				
Severity	Medium				
Affected	GoldBonusVault.distributeBonus() USDGBMinting.mint() external call				
Description	distributeBonus() performs external calls via GBB_TOKEN.safeTransfer(user, bonusAmount) inside a loop, without nonReentrant. While standard ERC20 transfers are not reentrant, tokens with hooks (ERC-777 style, or non-standard ERC20s) can reenter and cause unexpected behavior. Even if roles prevent direct reentrancy into privileged logic, hook-driven reentrancy can still create edge-case failures, griefing, or state inconsistencies in future upgrades.				
Comment	Add ReentrancyGuard and mark distributeBonus() as nonReentrant. Also consider moving transfers to the end or using pull-based claiming.				
Status	Pending				

08	CertificateStaking “approvedLenders” is unused Leverage eligibility ignores whitelist and can mislead integrators
Severity	Medium
Affected	CertificateStaking.approvedLenders, CertificateStaking.addLender(), CertificateStaking.removeLender(), CertificateStaking.getLeverageEligibility()
Description	The contract maintains <code>approvedLenders</code> but <code>getLeverageEligibility()</code> does not reference it. As written, any integrator can query and treat the response as authoritative regardless of lender approval. This is an integration hazard: the system appears to have a lender allowlist but does not enforce it in the only lending-facing function.
Comment	Either incorporate <code>approvedLenders[msg.sender]</code> into the eligibility check, or remove the mapping and admin functions to avoid false security assumptions.
Status	Pending

O9	withdrawEther can be DoS and lacks operational guardrails
Severity	Minor
Affected	GOLDBACKBONDBase.withdrawEther()
Description	withdrawEther() forwards all ETH to msg.sender using a low-level call. While this is not an obvious re-entrancy loss-of-funds vector (no state is updated after the call), it can be operationally blocked if the caller is a contract that reverts on receiving ETH, preventing ETH recovery unless an alternate ADMIN_ROLE EOA calls it. Additionally, it emits no event, reducing auditability, and the comment references a “nonReentrant modifier” that is not actually implemented, creating a documentation-to-code mismatch.
Recommendation	Emit an event on withdrawal. Remove misleading comments about nonReentrant or implement it consistently.
Status	Pending

10	Ownership and role admins are independent, may create a privilege-transfer foot-gun
Severity	Minor
Affected	All privileged controls split across <code>onlyOwner</code> (<i>LayerZero/OApp</i> surface) and <code>onlyRole(...)</code>
Description	Smart contract uses two independent authority systems: Owner-controlled permissions inherited from <i>LayerZero OApp/OFT</i> (for example, <code>onlyOwner</code> on <code>setMsgInspector</code>, and typically other <i>OApp</i> configuration), and role-controlled permissions (<code>DEFAULT_ADMIN_ROLE</code> / <code>ADMIN_ROLE</code> / <code>MINTER_ROLE</code> / <code>BURNER_ROLE</code>) in <i>GOLDBACKBONDBase</i>. Transferring ownership does not automatically transfer or revoke <i>AccessControl</i> roles, and changing roles does not change owner. This split frequently leads to deployments where a “former admin/owner” unintentionally retains some privileged capability (or, conversely, where the new governance cannot operate parts of the system).
Recommendation	Define a single source of authority, or enforce synchronized transitions: when ownership is transferred, also migrate/renounce <code>DEFAULT_ADMIN_ROLE</code> and <code>ADMIN_ROLE</code> to the same governance entity
Status	Pending

11	Hard limit is strict “<” instead of “<=”
Severity	Minor
Affected	USDGBMinting.mint()
Description	require(amount < hardLimit, "Exceeds Hard Limit"); rejects a mint equal to hardLimit. If the intended policy is “up to hard limit,” this is an off-by-one policy mismatch.
Recommendation	Change to require(amount <= hardLimit, "Exceeds Hard Limit"); if equality should be allowed.
Status	Pending

12	Limit invariants are not enforced (softLimit can exceed hardLimit)
Severity	Minor
Affected	USDGBMinting.setLimits()
Description	setLimits() does not enforce <code>_soft <= _hard</code> nor any sane minimum/maximum. Misconfiguration can unintentionally route most mints to the queue, or make the “green lane” unreachable.
Recommendation	Add <code>require(_soft <= _hard, "...")</code> and consider upper bounds consistent with token supply policy.
Status	 Pending

13	Approve/reject can mark non-existent queue entries as processed
Severity	Minor
Affected	USDGBMinting.approveTransaction(), USDGBMinting.rejectTransaction()
Description	If txId was never created, the mapping returns a default struct with <code>processed = false, user = address(0), amount = 0</code>. <code>approveTransaction()</code> will set <code>processed = true</code> and call <code>usdgb.mint(address(0), 0)</code> (usually harmless, but behavior depends on token implementation). This also allows emitting “approved/rejected” events for IDs that were never queued.
Recommendation	Store an explicit <code>exists</code> flag in <code>PendingTx</code> , or require <code>request.timestamp != 0</code> (or similar) to assert existence.
Status	Pending

14	Front-running possibilities
Severity	Minor
Affected	USDGBMinting.approveTransaction(), USDGBMinting.rejectTransaction()
Description	LpRewardPool rewards depend on <code>block.timestamp</code> and are claimable at any time; this is not a classic MEV surface, but validators can minimally skew timestamps and users can “claim right before withdraw” (already enforced) to maximize payout. No price-based AMM interactions exist in this code. GoldBonusVault distributes via admin-called batches; there is no user-triggered slippage-sensitive action. The primary manipulation vector is oracle integrity and staleness rather than mempool ordering.
Recommendation	Use commit-reveal scheme or apply transaction ordering to mitigate front-running
Status	Pending

15	Third-party dependencies
Severity	Unknown
Affected	Token contract
Description	Dependencies include <i>OpenZeppelin Ownable</i>, <i>AccessControl</i>, and <i>ERC20</i>, plus <i>LayerZero OFT / OFTCore / OApp / options</i> and <i>precrime simulator</i> components. The exact <i>LayerZero</i> version, the exact <i>OApp</i> owner model, and the complete set of inherited configuration methods are not fully shown; these materially affect trust boundaries (especially peer configuration and message validation). OFT / OFTCore / OApp / Codec Libraries from <i>LayerZero</i>: Reviewed for integration risks, trust boundaries, and override interactions; not re-audited line-by-line as novel code.
Recommendation	Because these components are not from Goldbackbond, overall oracle and token behavior has severity is unknown. Protocol's safety hinges on: - A well-designed, manipulation-resistant interfaces. - A trusted, manipulation-resistant <i>LayerZero OFT / OFTCore / OApp / options</i> and <i>precrime simulator</i> components. - Use of standard, audited ERC-20 tokens.
Status	Pending

4 . CENTRALIZATION

Centralization is one of the leading causes of smart contract-related asset losses. When a contract assigns critical powers to a privileged role—such as an `owner`, `admin`, or `designated controller`—the associated risk becomes elevated, especially if that role is tied to a single externally owned account (EOA). In many cases, privileged roles serve operational or safety functions, including:

- Emergency/Operational Controls
- Contract Configuration

4.1 Noteworthy Privileged Functions

Function	Role
<code>grantMinterRole(address)</code>	<code>onlyRole(ADMIN_ROLE)</code>
<code>revokeMinterRole(address)</code>	<code>onlyRole(ADMIN_ROLE)</code>
<code>grantBurnerRole(address)</code>	<code>onlyRole(ADMIN_ROLE)</code>
<code>revokeBurnerRole(address)</code>	<code>onlyRole(ADMIN_ROLE)</code>
<code>withdrawEther()</code>	<code>onlyRole(ADMIN_ROLE)</code>
<code>mint(address,uint256)</code>	<code>onlyRole(MINTER_ROLE)</code>
<code>burn(address,uint256)</code>	<code>onlyRole(BURNER_ROLE)</code>
<code>setMsgInspector(address)</code>	<code>onlyOwner</code> (inherited, LayerZero OFTCore)
Other OApp/OFT administrative configuration (peers/options/delegate)	<code>onlyOwner</code> (inherited; exact list not shown)
<code>USDGBMinting.mint()</code>	<code>MINTER_ROLE</code>
<code>USDGBMinting.approveTransaction()</code>	<code>COMPLIANCE_ADMIN</code>

Function	Role
USDGBMinting.rejectTransaction()	COMPLIANCE_ADMIN
USDGBMinting.setLimits()	DEFAULT_ADMIN_ROLE
USDGBMinting.pause()	DEFAULT_ADMIN_ROLE
USDGBMinting.unpause()	DEFAULT_ADMIN_ROLE
LpRewardPool.pause()	PAUSER_ROLE
LpRewardPool.unpause()	DEFAULT_ADMIN_ROLE
GoldBonusVault.distributeBonus()	DEFAULT_ADMIN_ROLE
GoldBonusVault.pause()	PAUSER_ROLE
GoldBonusVault.unpause()	DEFAULT_ADMIN_ROLE
CertificateStaking.addLender()	DEFAULT_ADMIN_ROLE
CertificateStaking.removeLender()	DEFAULT_ADMIN_ROLE
CertificateStaking.pause()	PAUSER_ROLE
CertificateStaking.unpause()	DEFAULT_ADMIN_ROLE
Guardian.emergencyGlobalPause()	RISK_MANAGER_ROLE / EXECUTIVE_ADMIN_ROLE
Guardian.restoreOperations()	EXECUTIVE_ADMIN_ROLE
Guardian.updateTargets()	EXECUTIVE_ADMIN_ROLE

01	Centralized Privileges
Severity	Centralization
Affected	Centralized and controlled roles
Description	EOAs with control can be compromised via phishing, private key leakage, or insider threats. Malicious or negligent use of privileges can lead to - token supply manipulation, disruption of trading via pausing, arbitrary fee changes or wallet exclusions, asset seizures or rerouting, etc. <u>4.1 Privileged Functions</u>
Recommendation	Using Multi-Signature Wallets: Assign privileged roles to a multi-sig contract requiring signatures from multiple trusted parties. This reduces the impact of any single compromised key. Time-Locked Functions: Introduce delays before executing sensitive operations, allowing time for community review or cancellation. Role Revocation or Transfer: If privileges are no longer needed post-deployment, renounce them or migrate them to DAO governance. Secure Key Management: Any private keys associated with privileged roles must be protected using hardware wallets, secret sharing schemes, or offline signing protocols.
Status	Pending

02	Owners & controlled roles can significantly manipulate the token contract
Severity	Centralization
Description	Privileged Minting via MINTER_ROLE Allows Supply Inflation up to MAX_SUPPLY - Any address with MINTER_ROLE can mint arbitrary amounts to any recipient, bounded only by MAX_SUPPLY on the local chain. This is a direct privileged supply control lever and must be treated as a centralized trust assumption (key compromise or malicious governance can inflate supply and harm holders). Privileged Burning from Arbitrary Accounts via BURNER_ROLE Is Confiscatory - Any address with BURNER_ROLE can burn tokens from any from address without allowance/approval. This is effectively a confiscation capability. Even if intended (e.g., compliance/redemption), it is a major user-trust and key-compromise risk (and can be weaponized for censorship or forced liquidation impacts). ADMIN_ROLE Can Sweep All Native ETH Held by the Contract - Any address with ADMIN_ROLE can transfer the full ETH balance to itself. Since the OFT send() flow is payable and may involve refunds/fees, ETH can accumulate in the contract (including via forced ETH). This becomes an admin-controlled sweep function. Owner-Controlled LayerZero Configuration Can Censor Bridging - OFTCore exposes at least setMsgInspector() guarded by onlyOwner. A message inspector can be used to block certain messages/options (censorship/DoS) and materially affect bridging UX. In typical LayerZero

OApp implementations, the owner also controls peer/trusted remote configuration; misconfiguration or compromise can expand/alter which remote senders are trusted, which can translate into unauthorized inbound mints on the destination chain (bounded by local MAX_SUPPLY, but still catastrophic for holders).

Recommendation	Governance hardening:	
	○ Treat the owners and controlled roles as highly privileged roles; use multisig + timelock; apply staged rollouts for peer changes; monitor config-change events; consider immutable config after launch where feasible.	
Status		Pending

03 Owners & controlled roles can significantly manipulate the mint contract	
Severity	Centralization
Description	Compliance admin can unilaterally approve queued mints (policy trust assumption) USDGBMinting.approveTransaction(), USDGBMinting.rejectTransaction() COMPLIANCE_ADMIN can approve any queued mint and cause token issuance. This is the intended design, but it is a single-role trust point that can be abused if compromised.
	Default admin can arbitrarily change mint limits and pause minting USDGBMinting.setLimits(), USDGBMinting.pause(), USDGBMinting.unpause() DEFAULT_ADMIN_ROLE can change limits to effectively disable minting, force all mints into compliance queue, or set extremely high limits (subject to the token’s own mint restrictions). Admin can also pause all minting.
Recommendation	Governance hardening: Treat the owners and controlled roles as highly privileged roles; use multisig + timelock; apply staged rollouts for peer changes; monitor config-change events; consider immutable config after launch where feasible.
Status	Pending

04	LpRewardPool pauser can halt staking/withdrawing/claiming CertificateStaking pause blocks withdrawals
Severity	Centralization
Affected	LpRewardPool.pause(), LpRewardPool.unpause(), all whenNotPaused functions CertificateStaking.pause(), CertificateStaking.withdraw()
Description	Pausing blocks stake, withdraw, and claimReward. This gives the pauser strong control over user LP accessibility (even if intended as emergency tooling). Even after the 365-day lock expires, withdrawals remain blocked while paused, enabling an admin/guardian to freeze user funds indefinitely.
Recommendation	Ensure the pauser is a multisig account; allow withdrawals while paused (emergency withdrawal pattern).
Status	 Pending

O5	GoldBonusVault admin fully controls if/when bonus distributions occur
Severity	Centralization
Affected	GoldBonusVault.distributeBonus()
Description	Only DEFAULT_ADMIN_ROLE can distribute bonuses, meaning distribution is discretionary and can be withheld. Users cannot claim (No permissionless claim pattern)
Recommendation	Use a claim-based mechanism (like merkle proofs) or publish distribution schedule constraints on-chain.
Status	☐ Pending

06	Guardian “risk bot” can pause all protocol components; try/catch can mask failures
Severity	Centralization
Affected	Guardian.emergencyGlobalPause(), Guardian.restoreOperations(), Guardian.updateTargets()
Description	RISK_MANAGER_ROLE (bot / third party) can pause all target contracts. Additionally, <code>try/catch {}</code> swallows errors, meaning <code>EmergencyGlobalPause</code> may emit even if one or more targets failed to pause; this can create a false sense of safety during incidents. <code>updateTargets()</code> can also repoint the guardian to arbitrary contracts.
Recommendation	○ Require multisig for risk actions or at least dual-control for pausing ○ Emit per-target success/failure events ○ Validate nonzero targets and use immutability or timelocks on target updates.
Status	Pending

5. DISCLAIMER

InterFi Network provides professional smart contract audits for blockchain-based codebases (commonly known as smart contracts). This audit assessed the reviewed contract(s) for common vulnerabilities, centralization risks, and logic flaws. However, no audit can guarantee the complete absence of bugs or vulnerabilities. This report does not constitute a security guarantee, endorsement, or assurance of business model soundness or legal compliance.

The review is limited strictly to the source code and its logic as provided, and does not extend to compiler behavior, off-chain components, or external integrations. Due to the evolving nature of blockchain technology and associated risks, users should understand that all materials, including this audit report, are provided strictly on an “as is”, “as available”, and “with all faults” basis.

5.1 Confidentiality

This report is confidential and intended solely for the client. It may not be disclosed, reproduced, or relied upon by third parties without prior written consent from InterFi Network. All terms, including confidentiality, liability limitations, and scope, are governed by the audit agreement.

5.2 No Financial Advice

This report is not financial, investment, tax, legal, or regulatory advice. It should not be relied upon for making investment decisions or assessing the value, viability, or safety of any token, product, or platform. No part of this document should be interpreted as an endorsement or recommendation. InterFi Network accepts no liability for any actions taken based on this report.

5.3 Technical Disclaimer

InterFi disclaims all warranties—express, implied, or statutory—including merchantability, fitness for a particular purpose, title, and non-infringement. We do not guarantee that the reviewed contracts are error-free, fully secure, or meet any specific requirements. Audit results may contain false positives or negatives, and findings are subject to the context and limitations of the review scope.

5.4 Timeliness & Accuracy

Audit results reflect the state of the code at the time of review. InterFi makes no commitment to update findings after publication. We do not warrant the accuracy, completeness, or timeliness of information delivered via this report.

5.5 Third-Party Links

This report may contain references or links to external websites and social media accounts. InterFi Network is not responsible for the content or operation of third-party platforms and assumes no liability for actions taken based on their content.

6. ABOUT

InterFi Network is a leading provider of intelligent blockchain solutions, offering secure, scalable, and production-ready smart contract services. Our team specializes in the development, testing, and auditing of smart contracts across a wide range of blockchain ecosystems.

We have delivered:

- 300+ smart contract systems developed
- 2,000+ smart contracts audited
- 500,000+ lines of code reviewed and analyzed

Our technical expertise spans multiple languages including:

- Solidity for EVM-compatible chains (Ethereum, BNB Chain, Polygon, Avalanche, Cronos, Fantom, Velas, Metis, and more)
- Move for next-generation platforms such as Sui and Aptos
- Rust for advanced ecosystems like Solana, Near, and Cosmos SDK-based chains

6.1 Connect with Us

InterFi Network is driven by a multidisciplinary team of engineers, developers, UI/UX specialists, and blockchain researchers. The core team consists of 3 senior members supported by 4+ expert contributors across code auditing, tooling, and protocol design.

- Website: interfi.network
- Email: hello@interfi.network
- GitHub: github.com/interfinetwork
- Telegram (Engineering): [@interfiaudits](https://t.me/interfiaudits)
- Telegram (Onboarding): [@interfisupport](https://t.me/interfisupport)

THANK YOU